Project Design Phase-II

Technology Stack (Architecture & Stack)

Project Detail	Value
Date	21 October 2026
Team ID	Individual Project
Project Name	StaffSync – Modern Human Resource Management System
Maximum Marks	4 Marks

Technical Architecture

StaffSync follows a three-tier web application architecture. The presentation layer is built with React, the application layer uses Node.js and Express.js to expose REST APIs and implement business logic, and the data layer uses MongoDB through Mongoose. Authentication and access control use JWT and bcryptjs. The frontend communicates with the backend through HTTP requests using Axios.

Table 1: Components & Technologies

S.N.	Component	Description	Technology
1	User Interface / Presentation Layer	Provides role-based screens for Employees, Managers, and HR, including dashboards, forms, tables, and status views.	React.js, JavaScript, HTML, CSS, Bootstrap / UI component styling as used in the project
2	Client-side Routing & State	Manages navigation, protected pages, shared application state, and API requests from the browser.	React Router, Redux Toolkit, Axios
3	Application Logic / REST API	Handles authentication, employee and manager management, attendance, leave workflows, announcements, training, performance, and notifications.	Node.js, Express.js, REST API
4	Database / Data Access	Stores user identities and HRMS records such as employees, managers, attendance, leaves, announcements, training, notifications, and performance.	MongoDB, Mongoose
5	Authentication & Security	Provides password hashing, token-based authentication, protected routes, role checks, request validation, and environment-based configuration.	JWT, bcryptjs, express-validator, dotenv, CORS, Helmet (where configured)
6	Development & Runtime Tools	Supports local development, automatic backend restart, frontend development server, and package management.	Node.js, npm, Vite, Nodemon, Git / GitHub

Table 2: Application Characteristics

S.N.	Characteristic	Description	Technology / Approach
1	Architecture Pattern	Separation of presentation, API/business logic, and data storage layers.	Three-tier architecture; MVC-style backend organization where implemented
2	API Communication	Frontend sends requests to backend endpoints and receives JSON responses.	HTTP / REST, JSON, Axios
3	Data Persistence	Stores application records in collections and uses schema/model definitions for data access.	MongoDB collections via Mongoose
4	Access Control	Restricts protected operations according to authenticated user role.	JWT authentication and role-based authorization middleware
5	Maintainability	Separates UI, services, API routes, controllers, models, and middleware for easier updates.	React component/service structure; Express routes/controllers/models
6	Configuration	Keeps environment-specific URLs and secrets outside source code.	.env configuration and .env.example templates

Technology Flow

Employee / Manager / HR → React frontend → Axios HTTP request → Express.js REST API on Node.js → authentication and authorization middleware → controller and Mongoose model → MongoDB. The API returns a JSON response to the frontend, which updates the relevant page or dashboard.

References

- React documentation: https://react.dev/
- Node.js documentation: https://nodejs.org/docs/latest/api/
- Express documentation: https://expressjs.com/
- MongoDB documentation: https://www.mongodb.com/docs/
- Mongoose documentation: https://mongoosejs.com/docs/
- JSON Web Token (JWT) introduction: https://jwt.io/introduction

Note: This document describes the StaffSync architecture and the technologies identified for the project. Confirm any optional UI library or middleware against the current package.json and source code before final submission.